Laporan Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027
Express 3D Container Packing



IF No Kelompok 43- Team KakAkuPakeAI:

-Hasna Nurul Alimah (11424061)
-Kholida Rezki Khoiriah (13222071)
-Hafizh Hananta Akbari (13522132)

DESKRIPSI PERSOALAN

Latar Belakang Kasus

Dalam industri logistik, pengangkutan barang menggunakan truk peti kemas membutuhkan efisiensi tinggi dalam penataan juga pemuatan paket. Industri ini dipenuhi dengan kendala fisik seperti keterbatasan volume ruang, daya angkut beban maksimum, serta sifat fisik barang seperti tingkat kerentanan barang pecah-belah yang secara khusus mempengaruhi efisiensi pengiriman. Oleh karena itu, dibutuhkan pendekatan algoritma local search untuk menentukan konfigurasi posisi, orientasi, dan pemilihan paket secara optimal agar total nilai ekonomi barang yang diangkut dapat dimaksimalkan tanpa melanggar batasan fisik yang ada.

Variabel Permasalahan

Permasalahan ini melibatkan entitas paket dan truk peti kemas. Setiap paket memiliki atribut berupa identitas unik (id), dimensi ukuran lebar, panjang, dan tinggi (w,l,h), serta orientasi rotasi pada sumbu x, y, atau z. Selain itu, paket dilengkapi dengan atribut nilai ekonomi (Value), berat barang (Weight), penanda status barang pecah-belah (IsFragile), serta perkiraan waktu kedatangan (ETA). Sementara itu, entitas truk peti kemas didefinisikan oleh atribut dimensi ruang dalam peti kemas (w,l,h) dan batas kapasitas daya angkut berat maksimum (Max Capacity).

Representasi State dan Neighbor

State direpresentasikan sebagai pemetaan posisi koordinat tiga dimensi (x,y,z) beserta orientasi dari seluruh paket yang ada, baik yang berhasil dimasukkan ke dalam truk maupun yang berada diluar peti kemas. Pencarian dimulai dari *initial state* yang dibentuk dengan menempatkan dan merotasi posisi paket secara acak. Transisi untuk menghasilkan solusi tetangga (*neighbor state*) pada setiap iterasi dilakukan melalui salah satu dari tiga bentuk pergerakan, yaitu menukar posisi dua paket, memindahkan satu paket ke lokasi koordinat baru, atau memutar orientasi paket sebesar 90° terhadap sumbu x, y, atau z.

Objective Function dan Constraints

Fungsi tujuan (*objective function*) dari permasalahan ini adalah memaksimalkan total akumulasi nilai (Value) dari seluruh paket yang dimuat di dalam peti kemas truk. Solusi yang dihasilkan dianggap valid apabila memenuhi empat batasan (*constraints*) utama. Pertama, seluruh paket di dalam truk harus berada penuh di dalam ruang peti kemas dan tidak boleh saling bertampalan. Kedua, paket tidak boleh melayang, yang berarti setiap alas paket wajib menyentuh lantai peti kemas ($z = 0$) atau berada tepat di atas permukaan paket lain yang menopangnya. Ketiga, paket pecah-belah (IsFragile = True) tidak boleh diletakkan di bawah paket lain atau dijadikan penopang beban. Keempat, total berat seluruh paket yang diangkut tidak boleh melebihi batas kapasitas beban maksimum (MaxCapacity) dari truk.

PEMBAHASAN

1. Implementasi Hill-Climbing- Steepest Ascent

Fungsi steepest ascent hill climbing berfungsi sebagai eksekutor utama algoritma. Fungsi ini menerima masukan berupa *initial state* penataan paket dan mengembalikan *final state* terbaik yang ditemukan beserta nilai *objective function*-nya. Di setiap iterasi, algoritma mengevaluasi seluruh

neighbor valid (hasil dari tukar posisi, pindah koordinat, atau rotasi paket) dan memilih *neighbor* dengan skor tertinggi. Jika skornya lebih baik dari *current state*, posisi diperbarui. Proses berhenti ketika sudah mencapai *local optima* (tidak ada *neighbor* yang lebih bagus) atau batas maksimum iterasi terpenuhi (Silvilestari, 2021). Di setiap iterasi, algoritma memilih *neighbor* yang paling besar diantara semua *neighbor* yang ada dari semua aksi yang valid (tukar posisi, pindah koordinat, atau rotasi paket). Dalam mencari *neighbor* terbesar, algoritma menggunakan fungsi-fungsi *tradeNeighbor*, *rotateNeighbor*, dan *relocateNeighbor* dan mengumpulkan daftar *neighbor* tersebut di fungsi *neighborList*. *Neighbor* akan selalu mengambil nilai yang tertinggi berdasarkan dari fungsi *stateValue* dan mengupdate skor hingga tidak ada *neighbor* yang lebih baik dari *state* pada saat ini. Untuk menjalankan algoritma, fungsi yang dijalankan adalah *initializeHC()*.

Source Code Snippet:

```
# hillclimbing_stepestaccent.py
import random
import time
from copy import deepcopy
from objects import (initialize, isOccupied, updateInside, fitting, truck)
from visualization import visualize


# Objects.py helper
def stateValue(ListofObjects):
    totalWeight = 0
    for obj in ListofObjects:
        if getattr(obj.Orientation, 'inside', False):
            totalWeight += obj.Weight
    return totalWeight


def candidateCoordinates(listofObjects, i):
    current = listofObjects[i]
    candidates = []

    maxWidth = 10
    maxLength = 20
    maxHeight = 5

    for x in range(maxWidth - current.Dimensions.width + 1):
        for y in range(maxLength - current.Dimensions.length + 1):
            z = 0
            coordinates = [x, y, z]

            if z + current.Dimensions.height <= maxHeight and not isOccupied(listofObjects,
current, coordinates)[0]:
                candidates.append(coordinates)

    return candidates


def safe_rotate(obj, axis, listofObjects):
    original_dims = (obj.Dimensions.width, obj.Dimensions.length, obj.Dimensions.height)

    if axis == "x":
        obj.Dimensions.length, obj.Dimensions.height = obj.Dimensions.height,
obj.Dimensions.length
    elif axis == "y":
        obj.Dimensions.width, obj.Dimensions.height = obj.Dimensions.height,
obj.Dimensions.width
    elif axis == "z":
        obj.Dimensions.width, obj.Dimensions.length = obj.Dimensions.length,
obj.Dimensions.width
    else:
        return False
```

```
    if isOccupied(listofObjects, obj, obj.Orientation.coordinates)[0]:
        obj.Dimensions.width, obj.Dimensions.length, obj.Dimensions.height = original_dims
        return False

    obj.retrace(listofObjects)
    return True


def safe_trade(obj1, obj2, listofObjects):
    if obj2 is None or obj2 is obj1:
        return False

    obj1.trade(obj2, listofObjects)
    return True


def safe_relocate(obj, setofCoordinates, listofObjects):
    if setofCoordinates is None or len(setofCoordinates) < 3:
        return False

    checkPlacement = "vertical" if setofCoordinates[2] >= 0 else "horizontal"

    if isOccupied(listofObjects, obj, setofCoordinates, checkPlacement)[0]:
        return False

    obj.relocate(setofCoordinates, listofObjects)
    return True


# Generate neighbors
def rotateNeighbors(ListofObjects):
    neighbors = []
    values = []
    operations = []

    for i in range(len(ListofObjects)):
        for axis in ["x", "y", "z"]:
            neighbor = deepcopy(ListofObjects)

            success = safe_rotate(neighbor[i], axis, neighbor)

            if not success:
                continue

            neighbors.append(neighbor)
            values.append(stateValue(neighbor))
            operations.append({"type": "rotate", "package": neighbor[i].id, "axis": axis})

    return neighbors, values, operations
```

```
def tradeNeighbors(ListofObjects):
    neighbors = []
    values = []
    operations = []

    for i in range(len(ListofObjects)):
        for j in range(i + 1, len(ListofObjects)):
            neighbor = deepcopy(ListofObjects)

            success = safe_trade(neighbor[i], neighbor[j], neighbor)

            if not success:
                continue

            neighbors.append(neighbor)
            values.append(stateValue(neighbor))
            operations.append({"type": "trade", "package1": neighbor[i].id, "package2":
neighbor[j].id})

    return neighbors, values, operations


def relocateNeighbors(ListofObjects, i):
    neighbors = []
    values = []
    operations = []

    coordinatesList = candidateCoordinates(ListofObjects, i)

    for coordinates in coordinatesList:
        neighbor = deepcopy(ListofObjects)
        old_coordinates = neighbor[i].Orientation.coordinates[:]

        success = safe_relocate(neighbor[i], coordinates, neighbor)

        if success:
            neighbors.append(neighbor)
            values.append(stateValue(neighbor))
            operations.append({"type": "relocate", "package": neighbor[i].id, "from":
old_coordinates, "to": coordinates[:]})

    return neighbors, values, operations


def neighborList(listofObjects, i):
    neighborRotate, valuesRotate, opsRotate = rotateNeighbors(listofObjects)
    neighborTrade, valuesTrade, opsTrade = tradeNeighbors(listofObjects)
    neighborRelocate, valuesRelocate, opsRelocate = relocateNeighbors(listofObjects, i)

    valid_neighbor = neighborRotate + neighborTrade + neighborRelocate
```

```
    valid_values = valuesRotate + valuesTrade + valuesRelocate
    valid_operations = opsRotate + opsTrade + opsRelocate

    return valid_neighbor, valid_values, valid_operations


# Algorithm

def hclimbing_stepestaccent(ListofObjects, i):
    current_state = deepcopy(ListofObjects)
    current_value = stateValue(current_state)

    langkah = 0

    while True:
        choose_neighbor, values, operations = neighborList(current_state, i)

        if not choose_neighbor:
            break

        best_value = max(values)
        indexMax = values.index(best_value)

        best_neighbor = choose_neighbor[indexMax]

        if best_value > current_value:
            current_state = best_neighbor
            current_value = best_value
            langkah += 1
        else:
            break

    return current_state, current_value, langkah


def initializeHC():
    t = initialize()
    listofObjects = t[0]

    # State Awal dan State Value Awal
    initialState = deepcopy(listofObjects)
    initialValue = stateValue(listofObjects)

    print("\n")
    print("STATE AWAL")
    print("State Value Awal:", initialValue)
    print("Detail State Awal:", initialState)
    print("\n")

    bestState = deepcopy(listofObjects)
    bestValue = initialValue
```

```
    bestSteps = 0

    for i in range(len(listofObjects)):
        resultState, resultValue, steps = hclimbing_stepestaccent(listofObjects, i)

        if resultValue > bestValue:
            bestState = deepcopy(resultState)
            bestValue = resultValue
            bestSteps = steps

    print("HASIL AKHIR")
    print("Jumlah Iterasi:", bestSteps)
    print("State Value Akhir:", bestValue)
    print("Detail State Akhir:", bestState)

    visualize(bestState)

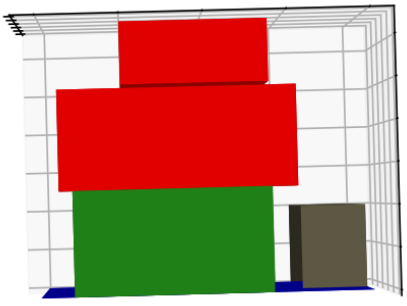
    return bestState, bestValue, bestSteps, initialState, initialValue


def testingHC():
    initializeHC()


if __name__ == "__main__":
    testingHC()
```

Visualisasi:
(initial state vs final state 3d)

Hasil Eksperimen

No	Hasil	Plot
1	State Awal= Jumlah Populasi (Paket): 4 State Value Awal: 138 Detail State Awal: [Package (ID = DqatPackage, (w = 7, l = 4, h = 4), (X: w, Y: l, Z: h, Coordinates = [3, 0, 0], Inside Truck: True), Value = 4, Weight = 88, IsFragile = False, ETA (day) = 4) , Package (ID = RopPackage, (w = 10, l = 12, h = 5), (X: w, Y: l, Z: h, Coordinates = [0, 6, 0], Inside Truck: True), Value = 73, Weight = 50, IsFragile = True, ETA (day) = 3) , Package (ID = DwcPackage, (w = 2, l = 15, h	

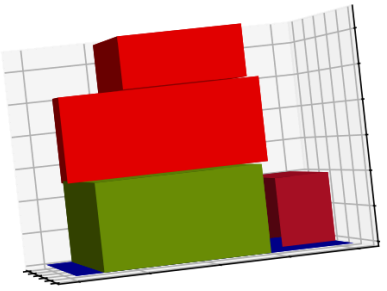
= 5), (X: w, Y: l, Z: h, Coordinates = [4, 4, 5], Inside Truck: False),
Value = 2, Weight = 2, IsFragile = False, ETA (day) = 5)

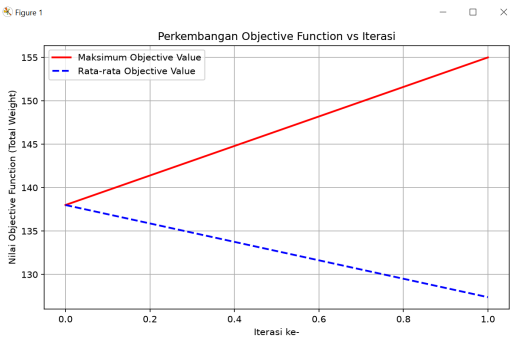
,
Package (ID = TjloPackage, (w = 7, l = 9, h = 3), (X: w, Y: l, Z: h, Coordinates = [1, 6, 10], Inside Truck: False),
Value = 7, Weight = 67, IsFragile = False, ETA (day) = 5)
]
Iterasi 1
  - Operasi   : {'type': 'trade', 'package1': 'RopPackage', 'package2': 'TjloPackage'}
  - Value Lama : 138 -> Value Baru: 155
Hasil Akhir=
Jumlah Populasi: 4
Jumlah Iterasi (Langkah): 1
State Value Akhir: 155
Detail Operasi Terbaik: [{'type': 'trade', 'package1': 'RopPackage', 'package2': 'TjloPackage'}]
Detail State Akhir: [
Package (ID = DqatPackage, (w = 7, l = 4, h = 4), (X: w, Y: l, Z: h, Coordinates = [3, 0, 0], Inside Truck: True),
Value = 4, Weight = 88, IsFragile = False, ETA (day) = 4)

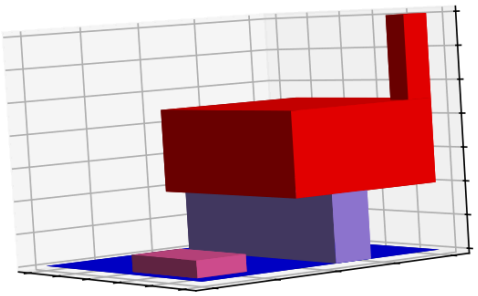
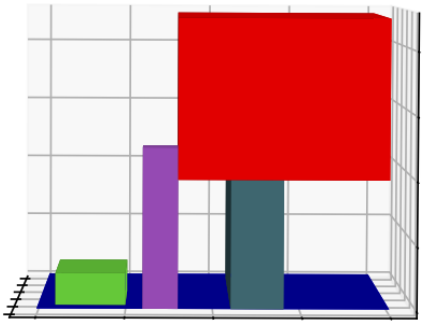
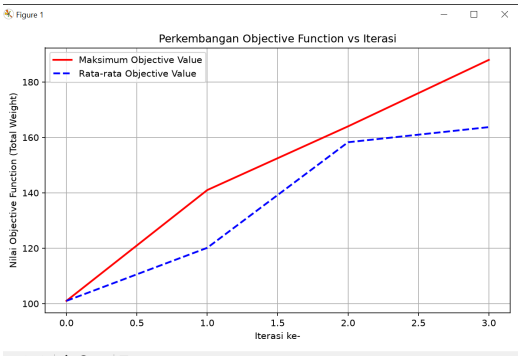
,
Package (ID = RopPackage, (w = 7, l = 9, h = 3), (X: w, Y: l, Z: h, Coordinates = [1, 6, 10], Inside Truck: False),
Value = 73, Weight = 50, IsFragile = True, ETA (day) = 3)

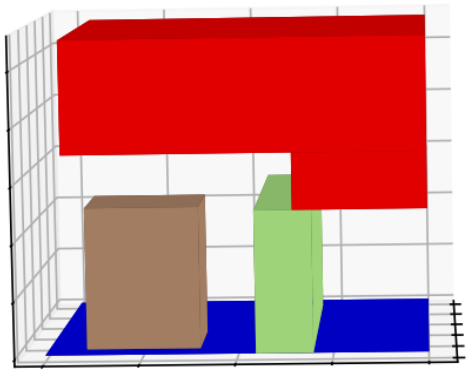
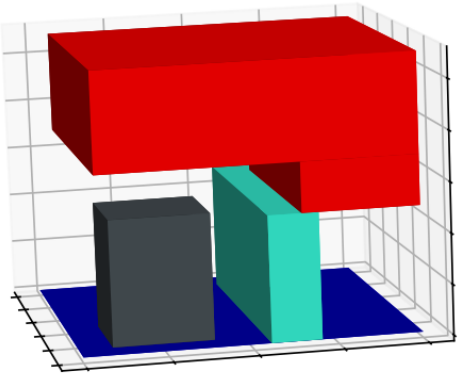
,
Package (ID = DwcPackage, (w = 2, l = 15, h = 5), (X: w, Y: l, Z: h, Coordinates = [4, 4, 5], Inside Truck: False),
Value = 2, Weight = 2, IsFragile = False, ETA (day) = 5)

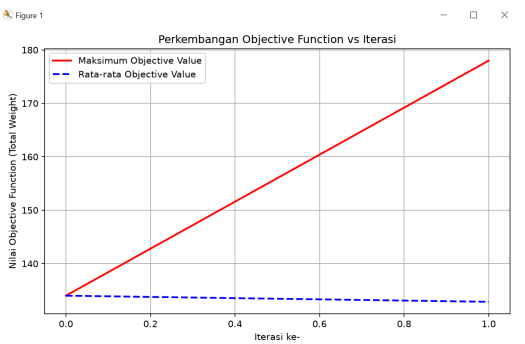
,
Package (ID = TjloPackage, (w = 10, l = 12, h = 5), (X: w, Y: l, Z: h, Coordinates = [0, 6, 0], Inside Truck: True),
Value = 7, Weight = 67, IsFragile = False, ETA (day) = 5)
]





2	State Awal= Jumlah Populasi (Paket): 4 State Value Awal: 101 Detail State Awal: [Package (ID = JssdPackage, (w = 9, l = 3, h = 4), (X: w, Y: l, Z: h, Coordinates = [0, 6, 0], Inside Truck: True), Value = 44, Weight = 47, IsFragile = False, ETA (day) = 3) , Package (ID = IcmnPackage, (w = 8, l = 12, h = 5), (X: w, Y: l, Z: h, Coordinates = [0, 0, 4], Inside Truck: False), Value = 51, Weight = 87, IsFragile = False, ETA (day) = 4) , Package (ID = MurtPackage, (w = 1, l = 2, h = 5), (X: w, Y: l, Z: h, Coordinates = [0, 0, 9], Inside Truck: False), Value = 14, Weight = 23, IsFragile = False, ETA (day) = 5) , Package (ID = VarsPackage, (w = 4, l = 4, h = 1), (X: w, Y: l, Z: h, Coordinates = [1, 15, 0], Inside Truck: True), Value = 91, Weight = 54, IsFragile = False, ETA (day) = 2)] Iterasi 1 - Operasi : {'type': 'trade', 'package1': 'JssdPackage', 'package2': 'IcmnPackage'} - Value Lama : 101 -> Value Baru: 141 Iterasi 2 - Operasi : {'type': 'relocate', 'package': 'MurtPackage', 'from': [0, 0, 9], 'to': [0, 12, 0]} - Value Lama : 141 -> Value Baru: 164 Iterasi 3 - Operasi : {'type': 'trade', 'package1': 'JssdPackage', 'package2': 'MurtPackage'} - Value Lama : 164 -> Value Baru: 188 Hasil Akhir= Jumlah Populasi: 4 Jumlah Iterasi (Langkah): 3 State Value Akhir: 188 Detail Operasi Terbaik: [{'type': 'trade', 'package1': 'JssdPackage', 'package2': 'IcmnPackage'}, {'type': 'relocate', 'package': 'MurtPackage', 'from': [0, 0, 9], 'to': [0, 12, 0]}, {'type': 'trade', 'package1': 'JssdPackage', 'package2': 'MurtPackage'}] Detail State Akhir: [Package (ID = JssdPackage, (w = 1, l = 2, h = 5), (X: w, Y: l, Z: h, Coordinates = [0, 12, 0], Inside Truck: True),	  

	Value = 44, Weight = 47, IsFragile = False, ETA (day) = 3) , Package (ID = IcmnPackage, (w = 9, l = 3, h = 4), (X: w, Y: l, Z: h, Coordinates = [0, 6, 0], Inside Truck: True), Value = 51, Weight = 87, IsFragile = False, ETA (day) = 4) , Package (ID = MurtPackage, (w = 8, l = 12, h = 5), (X: w, Y: l, Z: h, Coordinates = [0, 0, 4], Inside Truck: False), Value = 14, Weight = 23, IsFragile = False, ETA (day) = 5) , Package (ID = VarsPackage, (w = 4, l = 4, h = 1), (X: w, Y: l, Z: h, Coordinates = [1, 15, 0], Inside Truck: True), Value = 91, Weight = 54, IsFragile = False, ETA (day) = 2)]	
3	State Awal= Jumlah Populasi (Paket): 4 State Value Awal: 134 Detail State Awal: [Package (ID = MtmgPackage, (w = 3, l = 6, h = 5), (X: w, Y: l, Z: h, Coordinates = [1, 12, 0], Inside Truck: True), Value = 76, Weight = 54, IsFragile = False, ETA (day) = 1) , Package (ID = ScacPackage, (w = 9, l = 3, h = 5), (X: w, Y: l, Z: h, Coordinates = [0, 6, 0], Inside Truck: True), Value = 78, Weight = 80, IsFragile = False, ETA (day) = 0) , Package (ID = JsrPackage, (w = 8, l = 7, h = 2), (X: w, Y: l, Z: h, Coordinates = [0, 0, 5], Inside Truck: False), Value = 94, Weight = 98, IsFragile = False, ETA (day) = 0) , Package (ID = PisPackage, (w = 9, l = 19, h = 4), (X: w, Y: l, Z: h, Coordinates = [0, 0, 7], Inside Truck: False), Value = 16, Weight = 23, IsFragile = True, ETA (day) = 3)] Iterasi 1 - Operasi : {'type': 'trade', 'package1':	 

	'MtmgPackage', 'package2': 'JsrPackage'} - Value Lama : 134 -> Value Baru: 178 Hasil Akhir= Jumlah Populasi: 4 Jumlah Iterasi (Langkah): 1 State Value Akhir: 178 Detail Operasi Terbaik: [{'type': 'trade', 'package1': 'MtmgPackage', 'package2': 'JsrPackage'}] Detail State Akhir: [Package (ID = MtmgPackage, (w = 8, l = 7, h = 2), (X: w, Y: l, Z: h, Coordinates = [0, 0, 5], Inside Truck: False), Value = 76, Weight = 54, IsFragile = False, ETA (day) = 1) , Package (ID = ScacPackage, (w = 9, l = 3, h = 5), (X: w, Y: l, Z: h, Coordinates = [0, 6, 0], Inside Truck: True), Value = 78, Weight = 80, IsFragile = False, ETA (day) = 0) , Package (ID = JsrPackage, (w = 3, l = 6, h = 5), (X: w, Y: l, Z: h, Coordinates = [1, 12, 0], Inside Truck: True), Value = 94, Weight = 98, IsFragile = False, ETA (day) = 0) , Package (ID = PisPackage, (w = 9, l = 19, h = 4), (X: w, Y: l, Z: h, Coordinates = [0, 0, 7], Inside Truck: False), Value = 16, Weight = 23, IsFragile = True, ETA (day) = 3)]	

Berdasarkan hasil eksperimen, algoritma Hill Climbing Steepest Ascent (HC SA) dapat mencari dengan jumlah iterasi yang berbeda. Dalam 3 run berbeda, HC SA dua kali memiliki jumlah iterasi berupa 1 dan sekali dimana jumlah iterasinya 3. HC SA mencapai solusi yang valid dengan penataan paket yang valid dan dengan *state value* terbaik dari 3 run berbeda adalah 188. Jumlah iterasi yang bisa kecil menandakan algoritmanya yang cenderung berhenti dengan cepat karena sudah mencapai *local optima*.

2. Implementasi Simulated Annealing

Deskripsi fungsi dan logika utama:

Fungsi simulated annealing berfungsi sebagai eksekutor utama algoritma Simulated Annealing. Fungsi ini menerima masukan berupa *initial state* penataan paket, temperatur awal (*T*), dan *cooling rate*, lalu mengembalikan *state* terbaik yang berhasil ditemukan beserta nilai *objective function*-nya

(Guilmeau et al., 2021). Di setiap iterasi, algoritma memilih satu *neighbor* secara acak dari aksi pergerakan valid (tukar posisi, pindah koordinat, atau rotasi paket). Jika *neighbor* tersebut memiliki skor lebih baik, posisi langsung diperbarui. Jika skornya lebih buruk, *neighbor* tersebut masih bisa diterima berdasarkan nilai probabilitas ($e^{\Delta E/T}$) demi menghindari jebakan *local optima*. Temperatur (T) akan terus diturunkan secara bertahap sesuai *cooling rate* hingga mencapai temperatur minimum.
Logika Utama: Cooling schedule ($T_{next} = \alpha \cdot T$), perhitungan probabilitas penerimaan solusi lebih buruk $P(\Delta E) = e^{\Delta E/T}$, dan kondisi penghentian (T ≤ T_min).

Source Code Snippet:

```
import math, random, copy, time, os, sys
import matplotlib.pyplot as plt

# Parameter yang diubah tiap variasi
T0 = 50.0
alpha = 0.999
T_min = 1e-3
stuck_window = 20
SEED = 2

def simulated_annealing(problem, T0=50.0, alpha=0.999, T_min=1e-3, stuck_window=20):
    t0 = time.time()
    current = problem.initial_state()
    initial = problem.clone(current)
    E = problem.objective(current)
    best = problem.clone(current)
    bestE = E
    T = T0
    obj_hist, prob_hist = [], []
    stuck, rej_streak = 0, 0

    while T > T_min:
        nb = problem.neighbor(current)

        if not problem.feasible(nb):
            prob_hist.append(0.0)
            rej_streak += 1
        else:
            dE = problem.objective(nb) - E
            prob = 1.0 if dE > 0 else math.exp(dE / T)
            if random.random() < prob:
                current = nb
                E += dE
                rej_streak = 0
                if E > bestE:
                    best, bestE = problem.clone(current), E
            else:
                rej_streak += 1
            prob_hist.append(prob)
```

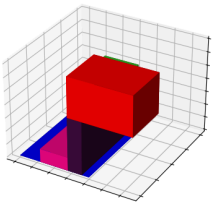
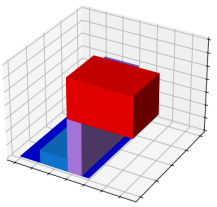
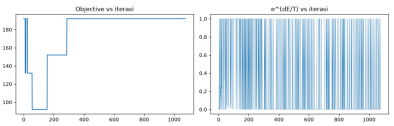
```
        if rej_streak == stuck_window:
            stuck += 1

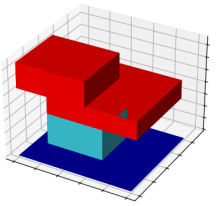
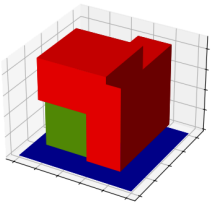
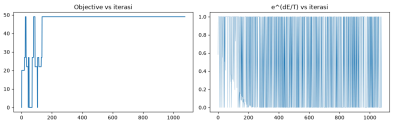
        obj_hist.append(E)
        T *= alpha

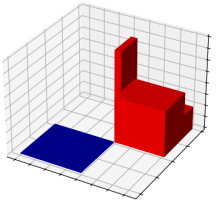
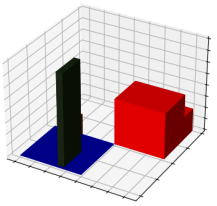
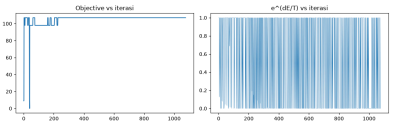
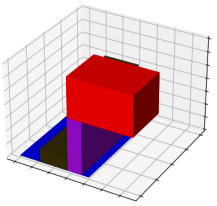
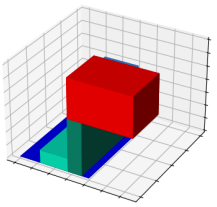
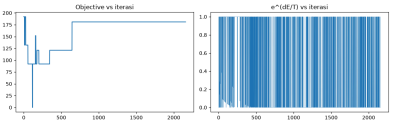
    return dict(
        initial=initial,
        final=best,
        final_value=bestE,
        obj_hist=obj_hist,
        prob_hist=prob_hist,
        iterations=len(obj_hist),
        stuck=stuck,
        duration=time.time() - t0,
    )
```

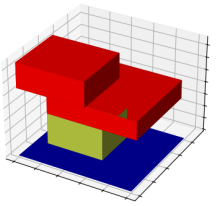
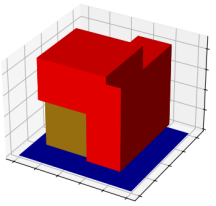
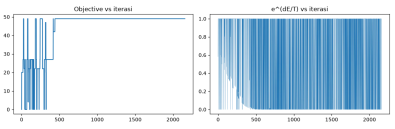
Visualisasi:
(initial state vs final state 3d)

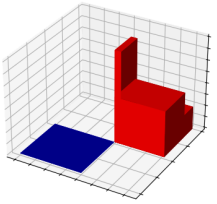
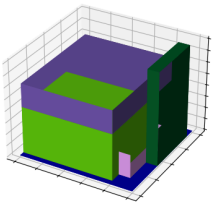
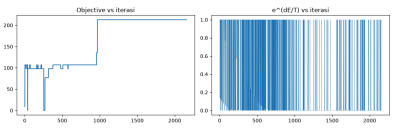
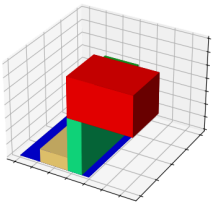
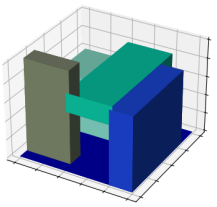
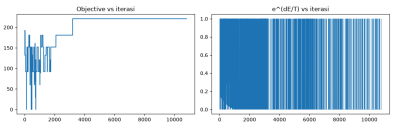
Hasil Eksperimen
Set 1: Variasi Banyak Iterasi (Populasi Kontrol/Tetap = 50)

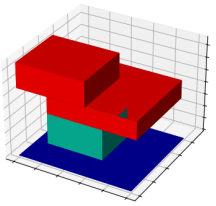
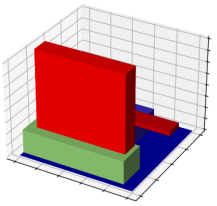
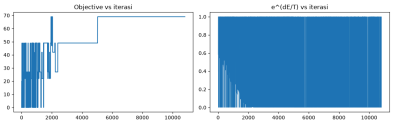
No	Populasi (N)	Alpha	Run Ke-	Hasil Eksperimen	SS / Plot
1	50	0.99	1	Parameter : T0=50.0, alpha=0.99, T_min=0.001, stuck_window=20, run=1 Banyak Iterasi : 1077 Durasi Eksekusi : 0.1567 detik State Awal : ['OubPackage', 'LtalPackage', 'VoqePackage'] State Akhir : ['OubPackage', 'LtalPackage', 'VoqePackage'] Objective Awal : 192 Objective Akhir : 192 Batas atas objective : 221 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  

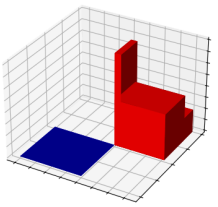
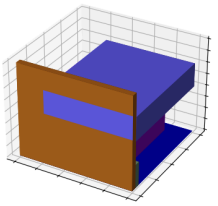
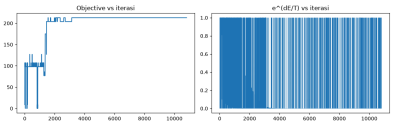
2	50	0.99	2	Parameter : T0=50.0, alpha=0.99, T_min=0.001, stuck_window=20, run=2 Banyak Iterasi : 1077 Durasi Eksekusi : 0.1610 detik State Awal : ['CjePackage'] State Akhir : ['CjePackage', 'XnogPackage'] Objective Awal : 27 Objective Akhir : 49 Batas atas objective : 73 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  

3	50	0.99	3	Parameter : T0=50.0, alpha=0.99, T_min=0.001, stuck_window=20, run=3 Banyak Iterasi : 1077 Durasi Eksekusi : 0.1507 detik State Awal : ['ZmvPackage'] State Akhir : ['IbxPackage', 'OxhgPackage'] Objective Awal : 0 Objective Akhir : 107 Batas atas objective : 213 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  
4	50	0.995	1	Parameter : T0=50.0, alpha=0.995, T_min=0.001, stuck_window=20, run=1 Banyak Iterasi : 2159 Durasi Eksekusi : 0.3166 detik State Awal : ['OubPackage', 'LtalPackage', 'VoqePackage'] State Akhir : ['OubPackage', 'LtalPackage', 'VoqePackage'] Objective Awal : 192 Objective Akhir : 192 Batas atas objective : 221 (total value semua paket) Frekuensi stuck : 1 Solusi valid : True	  

5	50	0.995	2	Parameter : T0=50.0, alpha=0.995, T_min=0.001, stuck_window=20, run=2 Banyak Iterasi : 2159 Durasi Eksekusi : 0.3358 detik State Awal : ['CjePackage'] State Akhir : ['CjePackage', 'XnogPackage'] Objective Awal : 27 Objective Akhir : 49 Batas atas objective : 73 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  

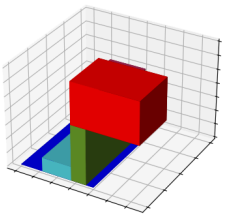
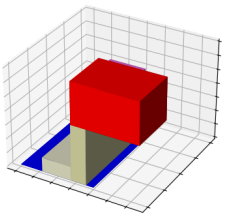
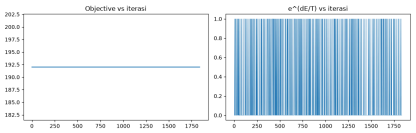
6	50	0.995	3	Parameter : T0=50.0, alpha=0.995, T_min=0.001, stuck_window=20, run=3 Banyak Iterasi : 2159 Durasi Eksekusi : 0.3251 detik State Awal : ['ZmvPackage'] State Akhir : ['PdxPackage', 'ZmvPackage', 'IbxPackage', 'OxhgPackage'] Objective Awal : 0 Objective Akhir : 213 Batas atas objective : 213 (total value semua paket) Frekuensi stuck : 20 Solusi valid : True	  
7	50	0.999	1	Parameter : T0=50.0, alpha=0.999, T_min=0.001, stuck_window=20, run=1 Banyak Iterasi : 10815 Durasi Eksekusi : 1.5446 detik State Awal : ['OubPackage', 'LtalPackage', 'VoqePackage'] State Akhir : ['OubPackage', 'LtalPackage', 'SntyPackage', 'VoqePackage'] Objective Awal : 192 Objective Akhir : 221 Batas atas objective : 221 (total value semua paket) Frekuensi stuck : 119 Solusi valid : True	  

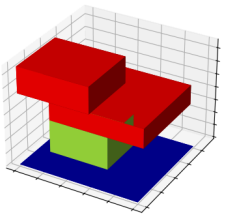
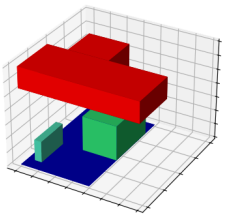
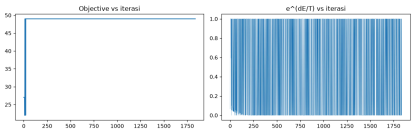
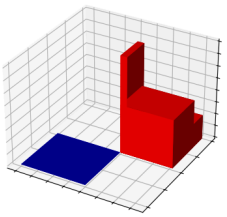
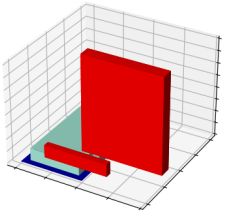
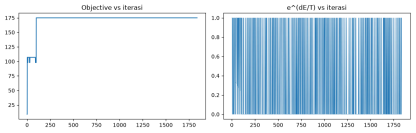
8	50	0.999	2	Parameter : T0=50.0, alpha=0.999, T_min=0.001, stuck_window=20, run=2 Banyak Iterasi : 10815 Durasi Eksekusi : 1.5495 detik State Awal : ['CjePackage'] State Akhir : ['CjePackage', 'XnogPackage', 'TxtuPackage'] Objective Awal : 27 Objective Akhir : 69 Batas atas objective : 73 (total value semua paket) Frekuensi stuck : 8 Solusi valid : True	  

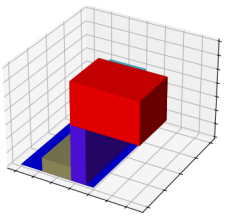
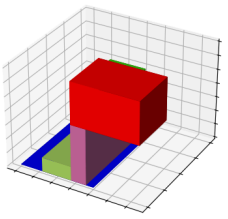
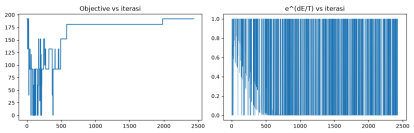
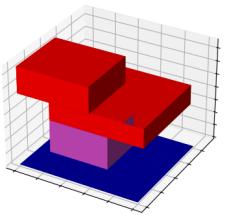
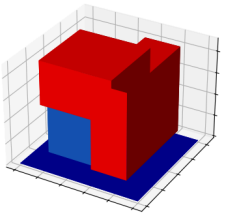
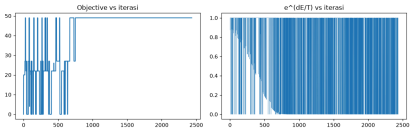
9	50	0.999	3	Parameter : T0=50.0, alpha=0.999, T_min=0.001, stuck_window=20, run=3 Banyak Iterasi : 10815 Durasi Eksekusi : 1.4815 detik State Awal : ['ZmvPackage'] State Akhir : ['PdxPackage', 'ZmvPackage', 'IbxPackage', 'OxhgPackage'] Objective Awal : 0 Objective Akhir : 213 Batas atas objective : 213 (total value semua paket) Frekuensi stuck : 147 Solusi valid : True	  

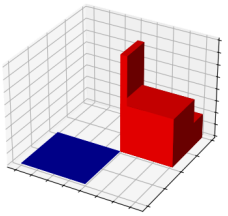
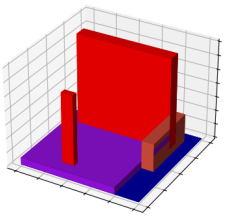
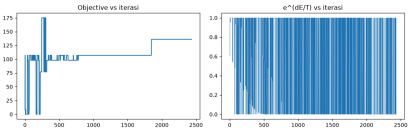
Set 2: Variasi Jumlah Populasi (alpha Kontrol/Tetap = 0.995)

No	Populasi	alpha	Run Ke-	Hasil Eksperimen	SS / Plot

10	10	0.995	1	Parameter : T0=10, alpha=0.995, T_min=0.001, stuck_window=20, run=1 Banyak Iterasi : 1838 Durasi Eksekusi : 0.2510 detik State Awal : ['OubPackage', 'LtalPackage', 'VoqePackage'] State Akhir : ['OubPackage', 'LtalPackage', 'VoqePackage'] Objective Awal : 192 Objective Akhir : 192 Batas atas objective : 221 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  

11	10	0.995	2	Parameter : T0=10, alpha=0.995, T_min=0.001, stuck_window=20, run=2 Banyak Iterasi : 1838 Durasi Eksekusi : 0.2567 detik State Awal : ['CjePackage'] State Akhir : ['CjePackage', 'XnogPackage'] Objective Awal : 27 Objective Akhir : 49 Batas atas objective : 73 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  
12	10	0.995	3	Parameter : T0=10, alpha=0.995, T_min=0.001, stuck_window=20, run=3 Banyak Iterasi : 1838 Durasi Eksekusi : 0.2526 detik State Awal : ['ZmvPackage'] State Akhir : ['PdxPackage', 'IbxPackage'] Objective Awal : 0 Objective Akhir : 175 Batas atas objective : 213 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  

13	200	0.995	1	Parameter : T0=200, alpha=0.995, T_min=0.001, stuck_window=20, run=1 Banyak Iterasi : 2436 Durasi Eksekusi : 0.3538 detik State Awal : ['OubPackage', 'LtalPackage', 'VoqePackage'] State Akhir : ['OubPackage', 'LtalPackage', 'VoqePackage'] Objective Awal : 192 Objective Akhir : 192 Batas atas objective : 221 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  
14	200	0.995	2	Parameter : T0=200, alpha=0.995, T_min=0.001, stuck_window=20, run=2 Banyak Iterasi : 2436 Durasi Eksekusi : 0.3650 detik State Awal : ['CjePackage'] State Akhir : ['CjePackage', 'XnogPackage'] Objective Awal : 27 Objective Akhir : 49 Batas atas objective : 73 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  

15	200	0.995	3	Parameter : T0=200, alpha=0.995, T_min=0.001, stuck_window=20, run=3 Banyak Iterasi : 2436 Durasi Eksekusi : 0.3494 detik State Awal : ['ZmvPackage'] State Akhir : ['PdxPackage', 'IbxPackage'] Objective Awal : 0 Objective Akhir : 175 Batas atas objective : 213 (total value semua paket) Frekuensi stuck : 0 Solusi valid : True	  

Tabel Ringkasan Rata-Rata

Set	Konfigurasi (alpha, populasi)	Rerata Obj Function Akhir	Best Obj Function Akhir	Rerata Durasi (s)	Rerata stuck (s)
1	alpha=0.99 (T0=50)	116	192	0.156	0
1	alpha=0.995 (T0=50)	151.33	213	0.326	7

1	alpha=0.999 (T0=50)	167.67	221	1.525	91.3
2	T0=10 (alpha=0.995)	138.67	192	0.253	0
2	T0=50 (alpha=0.995)	151.33	213	0.326	7
2	T0=200 (alpha=0.995)	138.67	192	0.356	0

Berdasarkan hasil eksperimen, Simulated Annealing (SA) mampu menemukan penataan paket yang valid pada seluruh run (solusi valid semua) dan mencapai skor *objective function* tertinggi 221 pada konfigurasi T0=50 dan alpha=0.999, yang sama dengan total value seluruh paket pada data tersebut. Peningkatan *cooling rate* (alpha) memberikan dampak paling signifikan terhadap kualitas solusi: rerata *objective function* naik dari 116,00 (alpha=0.99) menjadi 151,33 (alpha=0.995) dan 167,67 (alpha=0.999). Pendinginan yang lambat membuat SA bertahan lebih lama pada suhu tinggi, sehingga masih menerima langkah yang lebih buruk dan dapat menjelajahi lebih banyak konfigurasi sebelum menetap. Hal ini terlihat pada run 1, di mana hasil akhir pada alpha=0.99 dan alpha=0.995 tetap 192 (sama dengan *state* awal), dan hanya alpha=0.999 yang berhasil menaikkannya menjadi 221. Sebaliknya, alpha yang lebih besar menambah banyak iterasi (1.077, 2.159, dan 10.815) dan durasi eksekusi secara hampir linear (rerata 0,156, 0,326, dan 1,525 detik), dengan waktu per iterasi yang relatif sama. Frekuensi *stuck* juga ikut naik (rerata 0,0, 7,0, dan 91,3), terutama pada run yang sudah mencapai batas atas (seed 1 dan 3), karena setelah semua paket termuat hampir semua *neighbor* ditolak. Sementara itu, pengaruh temperatur awal (T0) pada alpha=0.995 tidak konsisten: T0=10 dan T0=200 sama-sama menghasilkan rerata 138,67, sedangkan T0=50 menghasilkan 151,33, selisih yang hanya berasal dari seed 3 (213 berbanding 175), sehingga dengan 3 run belum cukup untuk menyimpulkan pengaruh T0. Hal ini menunjukkan bahwa menurunkan suhu secara pelan (alpha besar) jauh lebih efektif untuk mendekati *global optima* dibandingkan sekadar mengubah T0, dengan konsekuensi durasi pencarian yang lebih lama.

3. Implementasi Genetic Algorithm

Deskripsi fungsi dan logika utama

Struktur kromosom atau genotipe pada program ini direpresentasikan sebagai sebuah *list* berisi sekumpulan objek paket (individual). Setiap gen di dalam kromosom menyimpan informasi spasial berupa posisi koordinat $[x, y, z]$ serta orientasi rotasi dari masing-masing paket. Kualitas penataan paket pada suatu kromosom dievaluasi menggunakan fitness_function(), yang mengkalkulasi

total nilai (*value*) dari seluruh paket yang berhasil muat secara valid di dalam truk (pkg.Orientation.inside).

Proses seleksi induk dilakukan oleh fungsi select_parents() dengan menerapkan metode *Roulette Wheel Selection*. Melalui metode ini, probabilitas suatu individu untuk terpilih sebanding dengan kontribusi nilai fitness yang dimilikinya terhadap total fitness seluruh populasi, sehingga individu dengan nilai fitness lebih tinggi memiliki peluang lebih besar untuk terpilih. Selanjutnya, fungsi crossover() menjalankan operasi silang satu titik (*single-point crossover*) untuk menggabungkan sifat dari dua induk. Sebuah titik potong acak dipilih sepanjang kromosom, lalu atribut orientasi dan posisi paket dari titik potong hingga akhir kromosom ditukarkan antara kedua induk guna membentuk dua individu anak (*offspring*). Setiap selesai persilangan, fungsi perbaikan fix_positions() dipanggil untuk memvalidasi penataan ulang posisi paket agar tidak saling tumpang tindih (*overlap*). Terakhir, fungsi mutate() memberikan variasi genetik pada setiap paket berdasarkan probabilitas mutasi $p_m = 0,1$. Jika mutasi terpicu, paket akan mengalami salah satu dari tiga aksi secara acak: memutar orientasi paket (rotate), mempertukarkan posisi dan orientasi dengan paket lain (trade), atau memindahkan paket ke koordinat acak baru (relocate).

Logika utama dari algoritma genetika ini dijalankan di dalam fungsi genetic_algorithm(). Algoritma diawali dengan membangkitkan populasi awal (N) individu secara acak melalui fungsi generate_initial_population(). Pada setiap generasi, nilai fitness seluruh individu dievaluasi dan dicatat untuk pemantauan performa nilai maksimum serta rata-rata populasi. Pembentukan populasi baru dilakukan sebanyak $\frac{N}{2}$ kali iterasi pasangan, di mana dua induk dipilih melalui proses seleksi, disilangkan untuk menghasilkan dua anak, dan dikenakan mutasi. Keturunan baru ini akan menggantikan populasi lama secara keseluruhan. Proses evolusi ini berulang hingga mencapai batas maksimum $T = 20$ generasi, kemudian algoritma mengembalikan populasi akhir beserta seluruh data perkembangan fitness.

Source Code Snippet:

```
import copy
import random
import time
from objects import initialize, pushAbove

# Parameter Utama GA
ukuran_populasi = 60      # Jumlah populasi (N)
jumlah_generasi = 30      # Banyak iterasi/generasi (T)
probabilitas_mutasi = 0.1

def fitness_function(chromosome):
    #Menghitung total nilai (value) dari paket yang berhasil muat secara valid di truk.
    total_value = 0
    for pkg in chromosome:
        if pkg.Orientation.inside:
            total_value += pkg.Value
    return total_value

def fix_positions(chromosome):
    #Validasi spasial untuk menyesuaikan posisi paket agar tidak melayang/overlap.
    for pkg in chromosome:
```

```
        pushAbove(chromosome, pkg)
        pkg.retrace(chromosome)
    return chromosome

def generate_initial_population():
    #Membangkitkan populasi awal secara acak.
    population = []
    for _ in range(ukuran_populasi):
        individual = copy.deepcopy(base_packages)
        for pkg in individual:
            pkg.rotate(random.choice(["x", "y", "z"]), individual)
            pkg.relocate([random.randint(0, 10), random.randint(0, 20), random.randint(0, 5)],
individual)
        individual = fix_positions(individual)
        population.append(individual)
    return population

def select_parents(population, fitness_scores):
    #Seleksi induk menggunakan metode Roulette Wheel Selection.
    total_fitness = sum(fitness_scores)
    if total_fitness == 0:
        selection_probs = [1 / len(population)] * len(population)
    else:
        selection_probs = [score / total_fitness for score in fitness_scores]
    return random.choices(population, weights=selection_probs, k=2)

def crossover(parent1, parent2):
    #Operasi silang satu titik (Single-point Crossover).
    child1, child2 = copy.deepcopy(parent1), copy.deepcopy(parent2)
    if len(parent1) > 1:
        crossover_point = random.randint(1, len(parent1) - 1)
        for i in range(crossover_point, len(parent1)):
            child1[i].Orientation = copy.deepcopy(parent2[i].Orientation)
            child2[i].Orientation = copy.deepcopy(parent1[i].Orientation)
            child1[i].retrace(child1)
            child2[i].retrace(child2)
    return fix_positions(child1), fix_positions(child2)

def mutate(chromosome):
    #Mutasi acak dengan probabilitas pm (rotate, trade, relocate).
    mutated = copy.deepcopy(chromosome)
    for pkg in mutated:
        if random.random() < probabilitas_mutasi:
            aksi = random.choice(["rotate", "trade", "relocate"])
            if aksi == "rotate":
                pkg.rotate(random.choice(["x", "y", "z"]), mutated)
            elif aksi == "trade" and len(mutated) > 1:
                target = random.choice([p for p in mutated if p != pkg])
                pkg.trade(target, mutated)
            elif aksi == "relocate":
                pkg.relocate([random.randint(0, 10), random.randint(0, 20), random.randint(0, 5)],
mutated)
    return fix_positions(mutated)
```

```
def genetic_algorithm():
    #Eksekutor utama siklus evolusi Genetic Algorithm.
    population = generate_initial_population()
    best_fitness_over_time, average_fitness_over_time = [], []

    for generation in range(jumlah_generasi):
        fitness_scores = [fitness_function(chromosome) for chromosome in population]
        best_fitness_over_time.append(max(fitness_scores))
        average_fitness_over_time.append(sum(fitness_scores) / len(fitness_scores))

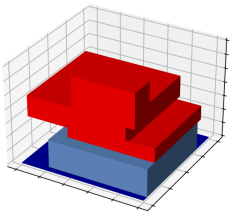
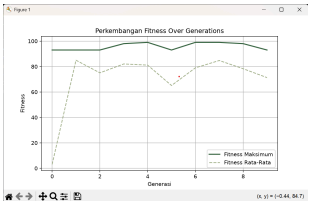
        new_population = []
        for _ in range(ukuran_populasi // 2):
            parent1, parent2 = select_parents(population, fitness_scores)
            child1, child2 = crossover(parent1, parent2)
            new_population.extend([mutate(child1), mutate(child2)])

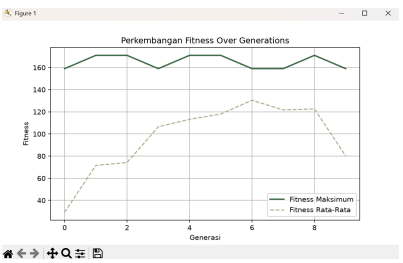
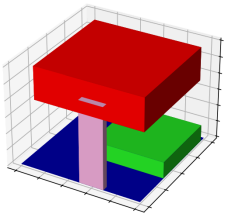
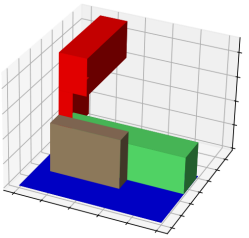
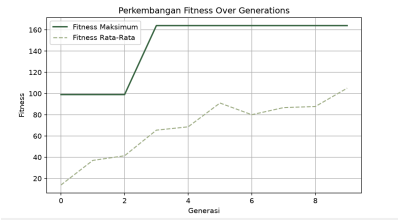
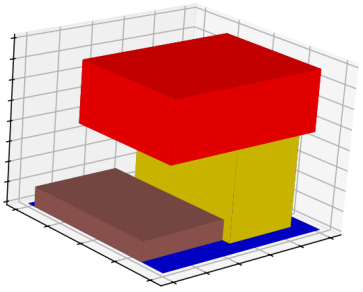
        population = new_population

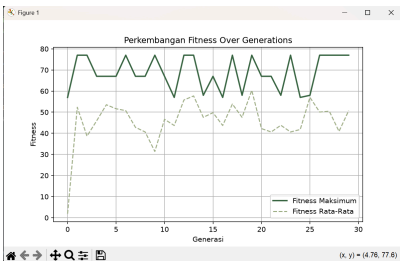
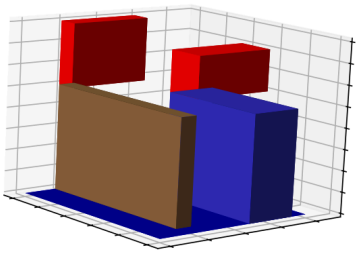
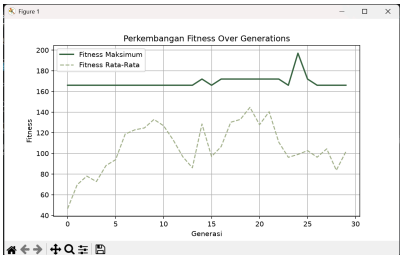
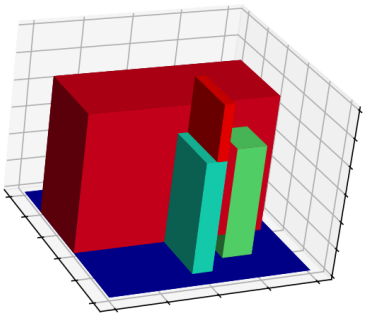
    return population, [best_fitness_over_time, average_fitness_over_time]
```

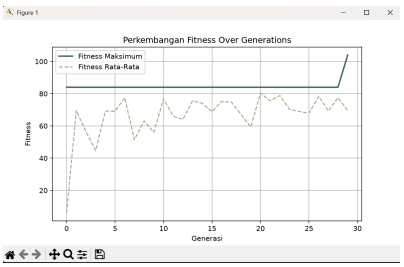
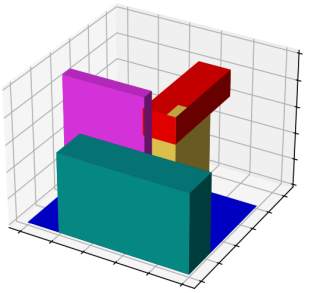
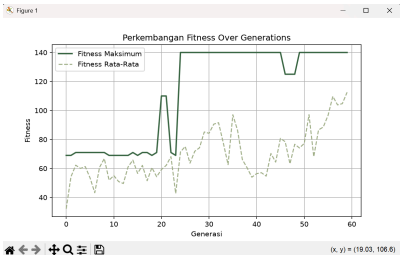
Hasil Eksperimen:

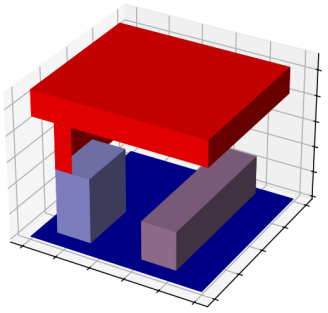
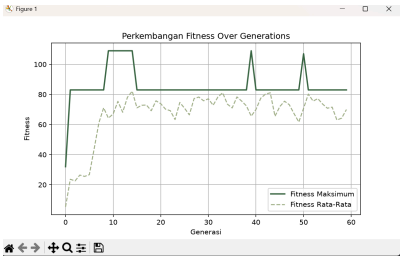
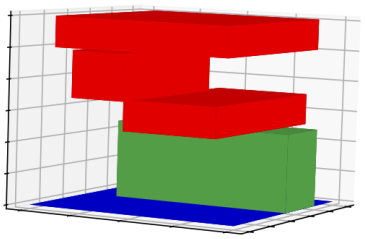
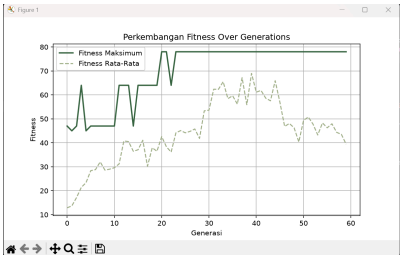
Set 1: Variasi Banyak Iterasi (Populasi Kontrol/Tetap = 30)

No	Populasi (N)	Iterasi (T)	Run Ke-	Hasil Eksperimen	SS / Plot
1	30	10	1	Jumlah Populasi : 30 Banyak Iterasi : 10 Durasi Eksekusi : 0.2216 detik Solusi Awal (State Awal) : ['WvqdPackage'] Solusi Terbaik Akhir (State Akhir) : ['AlxvPackage'] Nilai Objective Function Akhir : 99 / 10 Nilai Objective Function Rata-Rata : 87.00	 

2	30	10	2	Jumlah Populasi : 30 Banyak Iterasi : 10 Durasi Eksekusi : 0.2899 detik State Awal : ['WcxvPackage', 'YptvPackage'] State Akhir : ['WcxvPackage', 'YptvPackage'] Nilai Objective Function Akhir : 159 / 10 Nilai Objective Function Rata-Rata : 78.30	 
3	30	10	3	Jumlah Populasi : 30 Banyak Iterasi : 10 Durasi Eksekusi : 0.2015 detik State Awal : ['TiyPackage'] State Akhir : ['FeniPackage', 'TiyPackage'] Nilai Objective Function Akhir : 164 / 10 Nilai Objective Function Rata-Rata : 92.70	 
4	30	30	1	Jumlah Populasi : 30 Banyak Iterasi : 30 Durasi Eksekusi : 0.5699 detik State Awal : ['VakPackage'] State Akhir : ['AbgsPackage'] Nilai Objective Function Akhir : 67 / 10 Nilai Objective Function Rata-Rata : 44.37	

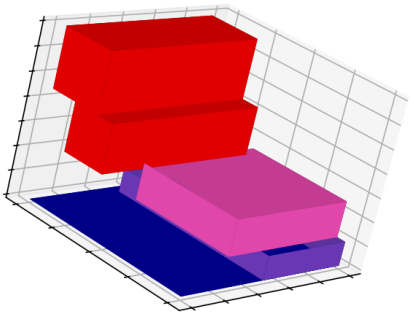
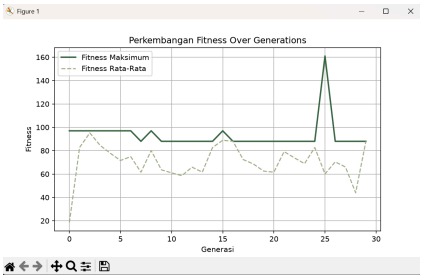
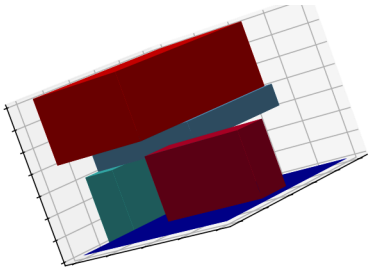
					
5	30	30	2	Jumlah Populasi : 30 Banyak Iterasi : 30 Durasi Eksekusi : 0.5219 detik State Awal : ['AadPackage', 'SfaePackage'] State Akhir : ['AadPackage', 'SfaePackage'] Nilai Objective Function Akhir : 166 / 10 Nilai Objective Function Rata-Rata : 103.43	 
6	30	30	3	Jumlah Populasi : 30 Banyak Iterasi : 30 Durasi Eksekusi : 0.6065 detik State Awal : ['OzxPackage'] State Akhir : ['BzgPackage', 'OzxPackage'] Nilai Objective Function Akhir : 104 / 10 Nilai Objective Function Rata-Rata : 61.33	

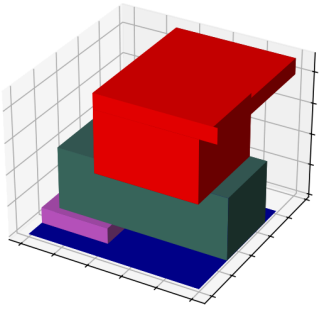
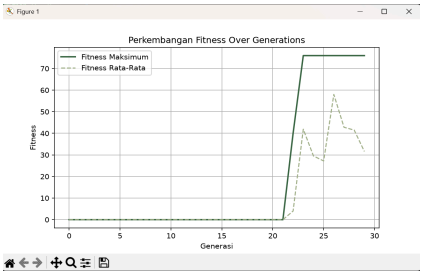
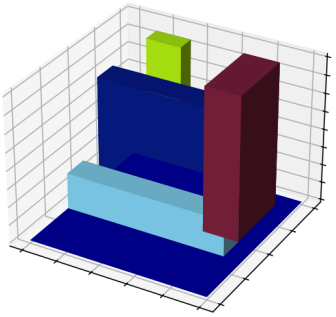
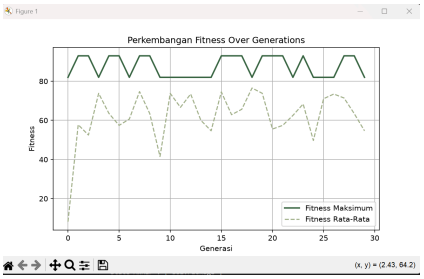
					
7	30	60	1	Jumlah Populasi : 30 Banyak Iterasi : 60 Durasi Eksekusi : 1.0574 detik State Awal : ['OjkPackage'] State Akhir : ['OjkPackage', 'BjjmPackage'] Nilai Objective Function Akhir : 140 / 10 Nilai Objective Function Rata-Rata : 84.73	 

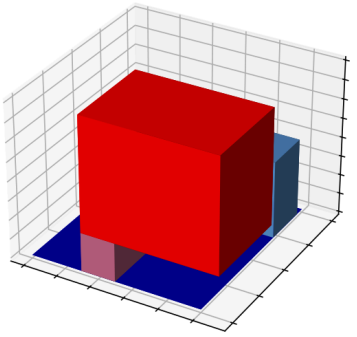
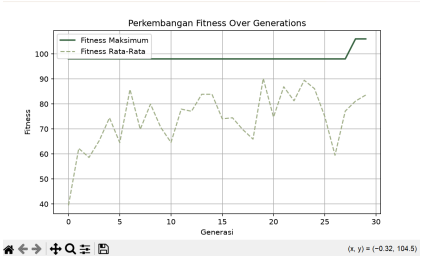
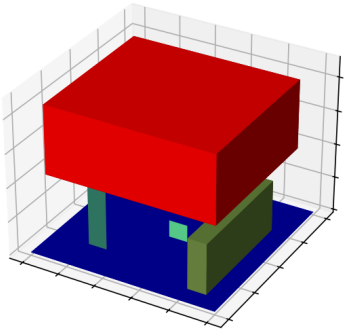
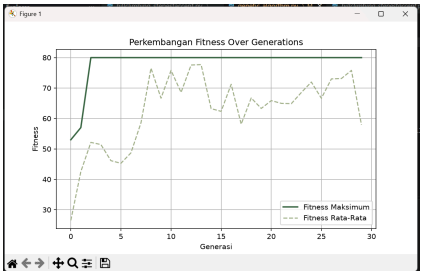
8	30	60	2	Jumlah Populasi : 30 Banyak Iterasi : 60 Durasi Eksekusi : 1.0002 detik State Awal : ['DhfrPackage'] State Akhir : ['PbydPackage'] Nilai Objective Function Akhir : 83 / 10 Nilai Objective Function Rata-Rata : 75.50	 
9	30	60	3	Jumlah Populasi : 30 Banyak Iterasi : 60 Durasi Eksekusi : 0.9910 detik State Awal : ['MoyPackage', 'TprPackage'] State Akhir : ['MoyPackage', 'TprPackage', 'YezhPackage'] Nilai Objective Function Akhir : 78 / 10 Nilai Objective Function Rata-Rata : 34.23	 

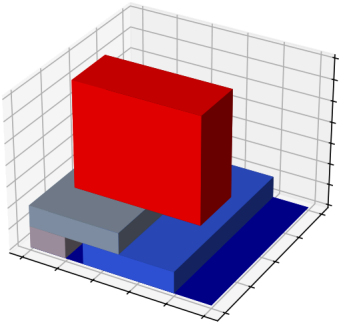
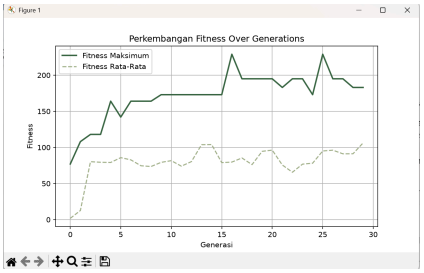
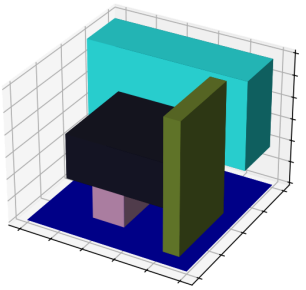
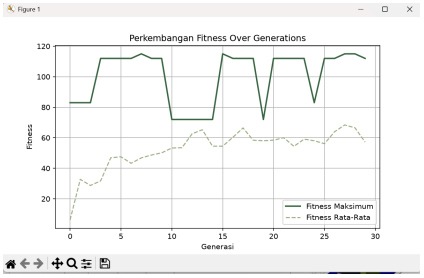
Set 2: Variasi Jumlah Populasi (Iterasi Kontrol/Tetap = 30)

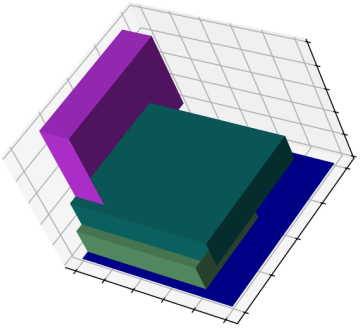
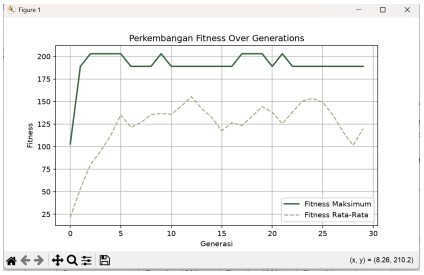
No	Populasi	Iterasi	Run Ke-	Hasil Eksperimen	SS / Plot

10	10	30	1	Jumlah Populasi : 10 Banyak Iterasi : 30 Durasi Eksekusi : 0.3322 detik State Awal : ['CwbPackage', 'EepPackage'] State Akhir :J ['CwbPackage'] Nilai Objective Function Akhir : 88 / 10 Nilai Objective Function Rata-Rata : 70.40	 
11	10	30	2	Jumlah Populasi : 10 Banyak Iterasi : 30 Durasi Eksekusi : 0.2091 detik State Awal : [] State Akhir : ['AnjPackage', 'BubpPackage'] Nilai Objective Function Akhir : 135 / 10 Nilai Objective Function Rata-Rata : 73.20	 

12	10	30	3	Jumlah Populasi : 10 Banyak Iterasi : 30 Durasi Eksekusi : 0.4606 detik State Awal : [] State Akhir : ['VcpyPackage'] Nilai Objective Function Akhir : 40 / 10 Nilai Objective Function Rata-Rata : 38.30	 
13	30	30	1	Jumlah Populasi : 30 Banyak Iterasi : 30 Durasi Eksekusi : 0.4730 detik State Awal : ['BsivPackage'] State Akhir : ['BsivPackage'] Nilai Objective Function Akhir : 82 / 10 Nilai Objective Function Rata-Rata : 77.07	 

14	30	30	2	Jumlah Populasi : 30 Banyak Iterasi : 30 Durasi Eksekusi : 0.5086 detik State Awal : ['IrgpPackage'] State Akhir : ['IrgpPackage'] Nilai Objective Function Akhir : 98 / 10 Nilai Objective Function Rata-Rata : 82.83	 
15	30	30	3	Jumlah Populasi : 30 Banyak Iterasi : 30 Durasi Eksekusi : 0.7541 detik State Awal : ['YmnPackage'] State Akhir : ['WgvrPackage'] Nilai Objective Function Akhir : 80 / 10 Nilai Objective Function Rata-Rata : 71.10	 

16	60	30	1	Jumlah Populasi : 60 Banyak Iterasi : 30 Durasi Eksekusi : 1.4580 detik State Awal : ['IuaPackage'] State Akhir : ['NwhzPackage', 'IuaPackage', 'ZzgkPackage'] Nilai Objective Function Akhir : 195 / 10 Nilai Objective Function Rata-Rata : 101.28	 
17	60	30	2	Jumlah Populasi : 60 Banyak Iterasi : 30 Durasi Eksekusi : 1.0656 detik State Awal : ['QldgPackage', 'XwxPackage'] State Akhir : ['MxraPackage', 'XwxPackage'] Nilai Objective Function Akhir : 115 / 10 Nilai Objective Function Rata-Rata : 61.88	 

18	60	30	3	Jumlah Populasi : 60 Banyak Iterasi : 30 Durasi Eksekusi : 1.0129 detik State Awal : ['XawPackage', 'NidPackage'] State Akhir : ['YiddPackage', 'XawPackage'] Nilai Objective Function Akhir : 189 / 10 Nilai Objective Function Rata-Rata : 115.60	 

Tabel Ringkasan Rata-Rata

Set	Konfigurasi (Pop, Iter)	Rerata Obj Function Akhir	Best Obj Function Akhir	Rerata Durasi (s)
Set 1	Pop: 30, Iter: 10	140.67	164	0.2377
Set 1	Pop: 30, Iter:30	112.33	166	0.5661
Set 1	Pop:30, Iter: 60	100.33	140	1.0162
Set 2	Pop: 10, Iter: 30	87.67	135	0.3340
Set 2	Pop: 30, Iter: 30	86.67	98	0.5786

Set 2	Pop: 60, Iter: 30	166.33	195	1.1788

Berdasarkan hasil eksperimen, Genetic Algorithm (GA) terbukti efektif dalam mengoptimalkan penataan paket 3D dengan skor *objective function* tertinggi mencapai 195 pada konfigurasi populasi 60 dan iterasi 30. Peningkatan jumlah populasi (N) memberikan dampak paling signifikan terhadap kualitas solusi meningkatkan rerata nilai *objective function* dari 87,67 menjadi 166,33 karena keragaman genetik yang tinggi mencegah populasi terjebak pada *local optima*. Sebaliknya, penambahan jumlah iterasi (T) hanya meningkatkan durasi eksekusi secara linear (sekitar 0,24 hingga 1,02 detik) tanpa peningkatan skor yang berarti jika populasi sudah terlanjur konvergen. Hal ini menunjukkan bahwa memperbesar ukuran populasi jauh lebih efektif untuk mengoptimalkan hasil pencarian dibandingkan sekadar menambah jumlah iterasi.

Analisis Hasil Eksperimen:

1. Seberapa dekat tiap-tiap algoritma bisa mendekati *global optima* dan mengapa hasilnya demikian?

   Simulated Annealing (SA) merupakan algoritma yang paling dekat dan berhasil mencapai global optima. Pada konfigurasi $\alpha = 0.999$ ($T_0 = 50$), SA mencapai Objective function sebesar 221 pada Run 1, yang setara dengan total nilai seluruh paket dalam dataset (batas atas maksimum). Hal ini disebabkan oleh mekanisme stokastik penerimaan solusi lebih buruk berdasarkan probabilitas Boltzmann ($P = e^{\frac{\Delta E}{T}}$). Di suhu tinggi, SA bebas melompati barrier penalti seperti IsFragile, penumpukan, dan kapasitas beban untuk keluar dari jebakan local optima, kemudian secara bertahap melakukan eksploitasi (fine-tuning) di suhu rendah.

   Hill-Climbing Steepest Ascent (HC SA) paling rentan terjebak di local optima lokal. Nilai terbaik yang ingin dicapai hanya 183 (pada Run1, terhenti di iterasi 0). Karena sifatnya yang murning greedy, HC SA langsung terhenti ketika tidak menemukan tetanga yang lebih baik. Lanskap permasalahan 3*D Container packing* memiliki banyak *plateau* dan *dead- end* akibat batasan ketat spasial dan fisik, sehingga HC SA gagal melakukan eksplorasi lebih jauh.

   Genetic Algorithm (GA) mampu mengeksplorasi ruang pencarian secara paralel melalui populasi kromosom, namun tingkat penderetam ke *global optimum* sangat bergantung pada efektivitas operator genetik. Pada kasus penataan 3D, operasi *crossover* sering kali merusak konfigurasi penataan yang sudah valid (memicu *overlap* atau paket melayang), sehingga memerlukan *repair function* (fix_positions()). Perbaikan paksa ini dapat menurunkan fleksibilitas eksplorasi, membuat GA memerlukan ukuran populasi dan generasi yang lebih besar untuk menyamai performa puncak SA.

2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma *local search* yang lain?

   SA secara signifikan mengungguli HC SA. Rerata *Objective Function* SA dengan $\alpha = 0.999$ mencapai 167.67, sedangkan HC SA kerap terhenti tanpa iterasi tambahan (rerata nilai akhir berkisar 128–183 tergantung *initial state*). SA mampu memperbaiki *initial state*

ber-skor 0 menjadi 213, sedangkan HC SA pada *initial state* ber-skor 104 hanya mampu naik menjadi 128 (1 iterasi).

SA menghasilkan solusi puncak (*peak value*) yang lebih tinggi daripada GA pada dataset pengujian yang sama. SA memproses transisi state secara atomik (*rotate*, *trade*, *relocate*) pada satu individu sehingga pencarian fokus dan efisien, sementara GA terbagi ke banyak individu yang membutuhkan konvergensi genetik bertahap.

GA unggul dari segi ketahanan eksplorasi karena memiliki variasi mutasi dan populasi, mencegah algoritma terhenti di iterasi ke-0 seperti pada HC SA. Namun, GA memiliki *overhead* perhitungan yang jauh lebih tinggi

3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?

   Hill- Climbing menujukkan durasi eksekusi paling singkat (di bawah 0.05–0.1 detik) dengan jumlah iterasi 0 hingga 1. Kecepatan ini dicapai karena algoritma langsung berhenti begitu berada di titik puncak lokal (*local optima*). Simulated Annealing sangat efisien dan terukur dengan durasi eksekusi berbandin lurus secara linear dengan laju pendinginan (α). Evaluasi per iterasi pada SA sangat ringan karena hanya menghitung perbedaan energi (ΔE) dari satu *neighbor* kandidat. Genetic Algorithm memiliki durasi terlama per iterasi. Pada konfigurasi $N = 60$ dan $T = 30$, durasi eksekusi mencapai 1.458 detik. *Overhead* tinggi ini disebabkan oleh kalkulasi fungsi *fitness* terhadap seluruh N individu di setiap generasi, proses seleksi *roulette wheel*, serta eksekusi validasi spasial (*fix_positions*) pasca-persilangan dan mutasi.

4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?

   Hasil HC SA cenderung deterministik terhadap intial state. Hasil akhir HC SA sangat bergantung pada keberuntungan *initial state*. Jika *initial state* berada di daerah yang terisolasi oleh batasan fisik, HC SA menghasilkan 0 iterasi, sehingga variansi solusinya murni mengikuti variansi pembuatan state acak awal. SA cenderung konsisten pada pendinginan lambat. Pada $\alpha = 0.999$, SA menunjukkan konsistensi tinggi dalam mencapai atau mendekati *global optima* (skor 221 dan 213 pada run yang diuji). Sebaliknya, pada $\alpha = 0.99$, pencarian terhenti terlalu cepat (*premature termination*) sehingga hasil akhirnya memiliki variansi yang tinggi. GA cenderung variansi moderat. Akibat sifat stokastik pada seleksi *roulette wheel* dan mutasi acak, hasil akhir GA pada jumlah generasi yang rendah ($T \leq 30$) menunjukkan variansi nilai akhir yang cukup lebar antarrun, sehingga memerlukan jumlah generasi yang cukup agar populasi mengalami konvergensi secara stabil.

5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

   Bertambahnya jumlah iterasi memberikan kesempatan bagi populasi untuk melakukan akumulasi gen-gen berkualitas tinggi (*building blocks*). Nilai *fitness* rata-rata populasi meningkat dan menstabilkan variansi antar-individu seiring berjalannya generasi. Namun, peningkatan *fitness* maksimum akan mengalami *diminishing returns* setelah populasi mulai mengalami konvergensi genetik. Ukuran populasi yang lebih besar ($N = 60$ vs $N = 10$) menjaga keanekaragaman genetik di awal generasi, mencegah *premature convergence* (terjebak di solusi lokal terlalu dini). Namun, peningkatan populasi berbanding lurus secara

linier terhadap durasi eksekusi, karena jumlah kalkulasi *fitness* dan operasi perbaikan posisi naik sebesar N kali lipat di setiap generasi. Kombinasi populasi sedang ($N = 30$) dengan iterasi yang cukup ($T \geq 60$) memberikan titik keseimbangan terbaik antara waktu komputasi dan kualitas penataan paket.

KESIMPULAN DAN SARAN

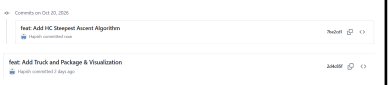
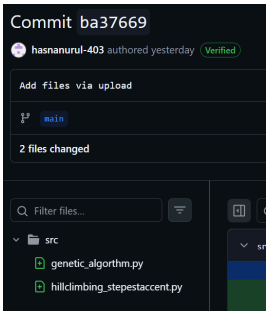
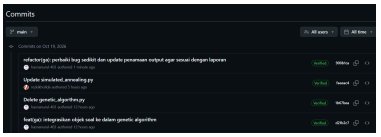
Kesimpulan

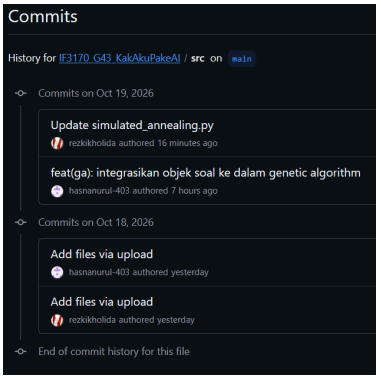
1. Simulated Annealing (SA) terbukti paling unggul dalam mendekati *global optima* (mencapai nilai objektif puncak 221 pada $\alpha = 0,999$) berkat mekanisme penerimaan solusi lebih buruk secara probabilistik yang mencegah jebakan *local optima* .
2. Hill-Climbing Steepest Ascent (HC SA) memiliki durasi eksekusi paling cepat (< 0,1 detik), namun sangat rentan terhenti prematur pada *local optima* dan hasil akhirnya sangat tergantung pada keberuntungan *initial state*.
3. Genetic Algorithm (GA) mampu melakukan eksplorasi secara paralel, tetapi membutuhkan durasi eksekusi paling tinggi per iterasi akibat kalkulasi *fitness* seluruh populasi dan eksekusi fungsi perbaikan posisi (fix_positions) pasca-persilangan/mutasi.

Saran

1. Kombinasikan Genetic Algorithm dan Simulated Annealing agar program bisa mencoba banyak variasi sekaligus mempercepat pencarian hasil terbaik tanpa mudah macet.
2. Atur penurunan suhu pada Simulated Annealing agar otomatis menyesuaikan kondisi jika pencarian mulai stuck, proses bisa dipercepat untuk menghemat waktu komputasi.

4. PEMBAGIAN KELOMPOK

Nama/ Nim	Pembagian Tugas	Bukti Branch
Hafizh Hananta Akbari	Objects, Visualization, Steepest Ascent Hill Climbing	
Hasna Nurul Alimah	Genetic Algorithm, raw code Steepest Ascent Hill Climbing	 

Kholida Rezki Khoiriah	Simulated Annealing	

5. REFERENSI

Silvilestari, S. (2021). Steepest Ascent Hill Climbing Algorithm To Solve Cases In Puzzle Game 8. *IJISTECH (International Journal of Information System and Technology)*, *5*(4), 366-370.

Guilmeau, T., Chouzenoux, E., & Elvira, V. (2021, July). Simulated annealing: A review and a new scheme. In *2021 IEEE statistical signal processing workshop (SSP)* (pp. 101-105). IEEE.

GeeksforGeeks. (2026, 10 Februari). *Genetic Algorithms*. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/genetic-algorithms/

GeeksforGeeks. (2026, 24 Agustus). *Hill Climbing in Artificial Intelligence*. GeeksforGeeks

6. LAMPIRAN FORM PENGGUNAAN AI

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM	: Hafizh Hananta Akbari/13522132 Hasna Nurul Alimah/11424061 Kholida Rezki Khoiriah/13222071
Fakultas/Sekolah	: Teknik Informatika/Institut Teknologi Bandung Rekayasa Pertanian/Institut Teknologi Bandung Teknik Elektro/Institut Teknologi Bandung
Kode Mata Kuliah	: IF3170
Nama Mata Kuliah	: Inteligensi Artifisial
Judul Tugas/Proposal	: Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	3
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	3
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Hafizh Hananta Akbari 13522132

7. LAMPIRAN LAINNYA